arXiv:1805.02867v2 [cs.PF] 28 Jul 2018

Online normalizer calculation for softmax

Maxim Milakov
NVIDIA
mmilakov@nvidia.com

Natalia Gimelshein
NVIDIA
ngimelshein@nvidia.com

Abstract

The Softmax function is ubiquitous in machine learning, multiple previous works suggested faster alternatives for it. In this paper we propose a way to compute classical Softmax with fewer memory accesses and hypothesize that this reduction in memory accesses should improve Softmax performance on actual hardware. The benchmarks confirm this hypothesis: Softmax accelerates by up to 1.3x and Softmax+TopK combined and fused by up to 5x.

1 Introduction

Neural networks models are widely used for language modeling, for tasks such as machine translation [1] and speech recognition [2]. These models compute word probabilities taking into account the already generated part of the sequence. The probabilities are usually computed by a Projection layer, which "projects" hidden representation into the output vocabulary space, and a following Softmax function, which transforms raw logits into the the vector of probabilities. Softmax is utilized not only for neural networks, for example, it is employed in multinomial logistic regression [3].

A number of previous works suggested faster alternatives to compute word probabilities. Differentiated Softmax [4] and SVD-Softmax [5] replace the projection layer - which is usually just a matrix multiplication - with more computationally efficient alternatives. Multiple variants of Hierarchical Softmax [6, 7, 8] split a single Projection+Softmax pair into multiple much smaller versions of these two functions organized in tree-like structures. Sampled-based approximations, such as Importance Sampling [9], Noise Contrastive Estimation [10], and Blackout [11] accelerate training by running Softmax on select elements of the original vector. Finally, Self-Normalized Softmax [12] augments the objective function to make the softmax normalization term close to 1 (and skip computing it during inference).

This is not an exhaustive list, but, hopefully, a representative one. Almost all of the approaches still need to run the original Softmax function, either on full vector or reduced one. There are two exceptions that don't need to compute the softmax normalization term: training with Noise Contrastive Estimation and inference with Self-Normalized Softmax. All others will benefit from the original Softmax running faster.

To the best of our knowledge there has been no targeted efforts to improve the performance of the original Softmax function. We tried to address this shortcoming and figured out a way to compute Softmax with fewer memory accesses. We benchmarked it to see if those reductions in memory accesses translate into performance improvements on a real hardware.

Preprint. Work in progress.

arXiv:1805.02867v2 [cs.PF] 28 Jul 2018

Softmax函数的在线归一化因子计算

Maxim Milakov,
NVIDIA,
mmilakov@nvidia.com

Natalia Gimelshein,
NVIDIA,
ngimelshein@nvidia.com

摘要

Softmax函数在机器学习领域中被广泛应用，此前已有不少研究提出了更为快速的替代方案。本文提出了一种能以更少的内存访问次数来计算传统Softmax函数的方法，并假设内存访问次数的减少有望提升该函数在实际硬件上的运行性能。实验结果证实了这一假设：单独使用Softmax函数的运算速度可提升至原来的1.3倍，而将Softmax函数与TopK算法结合并融合后，其运算速度甚至可提升至原来的5倍。

1 引言

神经网络模型被广泛用于语言建模，涵盖机器翻译 [1] 和语音识别 [2]等任务。这些模型在计算词的概率时会将已生成的序列部分纳入考量。通常，概率是通过投影层计算得出的，该层会将隐藏表示“投影”到输出词汇空间中，随后再通过Softmax函数将原始的对数几率转换为概率向量。Softmax函数不仅应用于神经网络，例如在多项逻辑回归 [3]中也会用到它。

此前已有不少研究提出了更快速的计算词概率的替代方法。差分Softmax [4] 与SVD-Softmax [5] 用计算效率更高的方案替代了通常仅为矩阵乘法的投影层。分层Softmax [6, 7, 8] 的多种变体则将单个投影+Softmax组合拆分成多个规模小得多的该函数版本，并以树状结构将其组织起来。基于采样的近似方法，如重要性采样 [9]、、噪声对比估计 [10]、以及黑掉法 [11]，通过仅对原始向量的部分元素应用Softmax函数来加速训练过程。最后，自归一化Softmax [12] 会对目标函数进行改进，使Softmax归一化项趋近于1（从而在推理阶段无需再计算该项）。

这份列表并非包罗万象，但希望能作为具有代表性的示例。几乎所有方法仍然需要对完整的向量或简化后的向量运行原始的Softmax函数。不过也有两种情况无需计算Softmax归一化项：采用噪声对比估计进行训练，以及使用自归一化Softmax进行推理。其余方法则都能从更快速的原始Softmax函数运算中受益。
据我们所知，目前尚无针对提升原始Softmax函数性能的专项优化工作。为解决这一缺陷，我们探索出了一种能够减少内存访问次数的Softmax计算方法，并通过基准测试验证了此类内存访问次数的减少是否真的能在实际硬件上带来性能提升。

预印本，仍在撰写中。

2 Original softmax

Function $y = Softmax(x)$ is defined as:

$$y_i = \frac{e^{x_i}}{\sum_{j=1}^{V} e^{x_j}} \tag{1}$$

where $x, y \in \mathbb{R}^V$. The naive implementation (see algorithm 1) scans the input vector two times - one to calculate the normalization term d_V and another to compute output values y_i - effectively doing three memory accesses per vector element: two loads and one store.

Algorithm 1 Naive softmax

1: $d_0 \leftarrow 0$
2: **for** $j \leftarrow 1, V$ **do**
3: $\quad d_j \leftarrow d_{j-1} + e^{x_j}$
4: **end for**
5: **for** $i \leftarrow 1, V$ **do**
6: $\quad y_i \leftarrow \frac{e^{x_i}}{d_V}$
7: **end for**

Unfortunately, on real hardware, where the range of numbers represented is limited, the line 3 of the algorithm 1 can overflow or underflow due to the exponent. There is a safe form of (1), which is immune to this problem:

$$y_i = \frac{e^{x_i - \max_{k=1}^{V} x_k}}{\sum_{j=1}^{V} e^{x_j - \max_{k=1}^{V} x_k}} \tag{2}$$

All major DL frameworks are using this safe version for the Softmax computation: TensorFlow

Algorithm 2 Safe softmax

1: $m_0 \leftarrow -\infty$
2: **for** $k \leftarrow 1, V$ **do**
3: $\quad m_k \leftarrow \max(m_{k-1}, x_k)$
4: **end for**
5: $d_0 \leftarrow 0$
6: **for** $j \leftarrow 1, V$ **do**
7: $\quad d_j \leftarrow d_{j-1} + e^{x_j - m_V}$
8: **end for**
9: **for** $i \leftarrow 1, V$ **do**
10: $\quad y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$
11: **end for**

[13] v1.7, PyTorch [14] (with Caffe2) v0.4.0, MXNET [15] v1.1.0, Microsoft Cognitive Toolkit [16] v2.5.1, and Chainer [17] v5.0.0a1. But Safe Softmax does three passes over input vector: The first one calculates the maximum value m_V, the second one - normalization term d_V, and the third one - final values y_i, see algorithm 2; This results in 4 memory access per vector element overall. We want to improve on that.

3 Online normalizer calculation

The algorithm 3 calculates both the maximum value m and the normalization term d in a single pass over input vector with negligible additional cost of two operations per vector element. It reduces memory accesses from 4 down to 3 per vector element for the Softmax function evaluation. Inspiration came from the numerically stable variance calculation online algorithm, see [18].

2 原始Softmax函数

函数 $y = Softmax(x)$的定义如下：

$$y_i = \frac{e^{x_i}}{\sum_{j=1}^{V} e^{x_j}} \tag{1}$$

其中 x、 $y \in \mathbb{R}^V$。那种简单的实现方式（见算法1）会遍历输入向量两次——一次用于计算归一化项 d_V，另一次用于得出输出值 y_i ——实际上每个向量元素需要执行三次内存访问：两次读取和一次写入。

算法1：朴素Softmax函数

1: $d_0 \leftarrow 0$2: 对于 $j \leftarrow 1, V$ 执行3: $d_j \leftarrow d_{j-1} + e^{x_j}$4: 结束循环5: 对于 $i \leftarrow 1, V$ 执行6: $y_i \leftarrow \frac{e^{x_i}}{d_V}$7: 结束循环

遗憾的是，在实际硬件上，由于所能表示的数值范围有限，算法1中的第3行可能会因为指数运算而出现上溢或下溢现象。存在一种经过改良的(1)形式，能够避免这一问题：

$$y_i = \frac{e^{x_i - \max_{k=1}^{V} x_k}}{\sum_{j=1}^{V} e^{x_j - \max_{k=1}^{V} x_k}} \tag{2}$$

所有主流的深度学习框架在计算Softmax函数时都在使用这一安全版本：TensorFlow

算法2：安全软缩放算法

1: $m_0 \leftarrow -\infty$2: 对于 $k \leftarrow 1, V$，执行3: $m_k \leftarrow \max(m_{k-1}, x_k)$4: 结束循环5: $d_0 \leftarrow 0$6: 对于 $j \leftarrow 1, V$，执行7: $d_j \leftarrow d_{j-1} + e^{x_j - m_V}$8: 结束循环9: 对于 $i \leftarrow 1, V$，执行10: $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$11: 结束循环

[13] v1.7、PyTorch [14] （含Caffe2）v0.4.0、MXNET [15] v1.1.0、Microsoft Cognitive Toolkit[16] v2.5.1以及Chainer [17] v5.0.0a1。不过安全软缩放算法会对输入向量进行三次处理：第一次计算最大值 m_V，第二次计算归一化系数 d_V，第三次计算最终结果 y_i，详见算法2；这样一来，每个向量元素总共需要4次内存访问。我们希望对此进行优化。

3 个在线归一化因子计算方法

算法3通过对输入向量进行单次遍历即可同时计算出最大值 m 与归一化系数 d，每个向量元素仅需执行两次操作，因此额外开销可忽略不计。此外，该算法还将用于计算Softmax函数的每个向量元素所需的内存访问次数从4次减少到了3次。这一设计的灵感源自数值上更为稳定的方差计算在线算法，相关内容可见 [18]。

Algorithm 3 Safe softmax with online normalizer calculation

1: $m_0 \leftarrow -\infty$
2: $d_0 \leftarrow 0$
3: **for** $j \leftarrow 1, V$ **do**
4: $\quad m_j \leftarrow \max\left(m_{j-1}, x_j\right)$
5: $\quad d_j \leftarrow d_{j-1} \times e^{m_{j-1}-m_j} + e^{x_j-m_j}$
6: **end for**
7: **for** $i \leftarrow 1, V$ **do**
8: $\quad y_i \leftarrow \frac{e^{x_i-m_V}}{d_V}$
9: **end for**

Essentially, the algorithm keeps the maximum value m and the normalization term d as it iterates over elements of the input array. At each iteration it needs to adjust the normalizer d to the new maximum m_j and only then add new value to the normalizer.

Theorem 1. *The lines 1-6 of the algorithm 3 compute* $m_V = \max_{k=1}^{V} x_k$ *and* $d_V = \sum_{j=1}^{V} e^{x_j - m_V}$

Proof. We will use a proof by induction.

♢ *Base case*: $V = 1$

$$
\begin{aligned}
m_1 &\leftarrow x_1 && \text{by line 4 of the algorithm 3}\\
&= \max_{k=1}^{1} x_k\\
d_1 &\leftarrow e^{x_1-m_1} && \text{by line 5 of the algorithm 3}\\
&= \sum_{j=1}^{1} e^{x_j-m_1}
\end{aligned}
$$

The theorem holds for $V = 1$.

♢ *Inductive step*: We assume the theorem statement holds for $V = S-1$, that is the lines 1-6 of the algorithm 3 compute $m_{S-1} = \max_{k=1}^{S-1} x_k$ and $d_{S-1} = \sum_{j=1}^{S-1} e^{x_j - m_{S-1}}$. Let's see what the algorithm computes for $V = S$

$$
\begin{aligned}
m_S &\leftarrow \max\left(m_{S-1}, x_S\right) && \text{by line 4 of the algorithm 3}\\
&= \max(\max_{k=1}^{S-1} x_k, x_S) && \text{by the inductive hypothesis}\\
&= \max_{k=1}^{S} x_k\\
d_S &\leftarrow d_{S-1} \times e^{m_{S-1}-m_S} + e^{x_S-m_S} && \text{by line 5 of the algorithm 3}\\
&= \left(\sum_{j=1}^{S-1} e^{x_j-m_{S-1}}\right) \times e^{m_{S-1}-m_S} + e^{x_S-m_S} && \text{by the inductive hypothesis}\\
&= \sum_{j=1}^{S-1} e^{x_j-m_S} + e^{x_S-m_S}\\
&= \sum_{j=1}^{S} e^{x_j-m_S}
\end{aligned}
$$

The inductive step holds as well. □

The algorithm 3 is proved to compute the Softmax function as defined in (2). It is also safe:

- m_j is the running maximum, $m_j \in \left[\min_{k=1}^{V} m_k, \max_{k=1}^{V} m_k\right], \forall j \in 1, V$; m_j cannot underflow or overflow.

算法3：结合在线归一化因子计算的安全软缩放算法

1: $m_0 \leftarrow -\infty$2: $d_0 \leftarrow 0$3: 对于 $j \leftarrow 1,$ V 执行4: $m_j \leftarrow \max\left(m_{j-1}, x_j\right)$5: $d_j \leftarrow d_{j-1} \times e^{m_{j-1}-m_j} + e^{x_j-m_j}$6: 结束循环7: 对于 $i \leftarrow 1, V$ 执行8: $y_i \leftarrow \frac{e^{x_i-m_V}}{d_V}$9: 结束循环

本质上，该算法在遍历输入数组的各个元素时，会持续记录最大值 m 以及归一化项 d。在每次迭代中，它都需要先将归一化因子 d 调整为新的最大值 m_j，之后才能将新数值加入该归一化因子中。

定理1：算法3中的第1至6行用于计算 $m_V = {}^{V}\max_{k=1}{}^{x_k}$ 以及 $d_V = \sum_{j=1}^{V} e^{x_j - m_V}$。

证明：我们将采用归纳法证明这一结论。

♢ *Base case*: $V = 1$

$$
\begin{aligned}
m_1 &\leftarrow x_1 && \text{依据算法3的第4行。}\\
&= \max_{k=1}^{1} x_k\\
d_1 &\leftarrow e^{x_1-m_1} && \text{依据算法3的第5行。}\\
&= \sum_{j=1}^{1} e^{x_j-m_1}
\end{aligned}
$$

该定理对 $V = 1$成立。

♢归纳步骤：我们假设该定理对 $V = S-1$成立，也就是在算法3的第1至6行中会计算 $m_{S-1} = {}^{S-1}\max_{k=1}{}^{x_k}$ 和 $d_{S-1} = \sum_{j=1}^{S-1} e^{x_j - m_{S-1}}$。接下来我们来看看该算法会为 $V = S$计算出什么结果。

$$
\begin{aligned}
m_S &\leftarrow \max\left(m_{S-1}, x_S\right) && \text{通过算法3的第4行。}\\
&= \max(\max_{k=1}^{S-1} x_k, x_S) && \text{依据归纳假设。}\\
&= \max_{k=1}^{S} x_k\\
d_S &\leftarrow d_{S-1} \times e^{m_{S-1}-m_S} + e^{x_S-m_S} && \text{通过算法3的第5行。}\\
&= \left(\sum_{j=1}^{S-1} e^{x_j-m_{S-1}}\right) \times e^{m_{S-1}-m_S} + e^{x_S-m_S} && \text{依据归纳假设。}\\
&= \sum_{j=1}^{S-1} e^{x_j-m_S} + e^{x_S-m_S}\\
&= \sum_{j=1}^{S} e^{x_j-m_S}
\end{aligned}
$$

归纳步骤同样成立。 □

经证明，算法3能够按照(2)中的定义计算Softmax函数，且该算法是安全的。

- m_j即为运行中的最大值，$m_j \in \left[{}^{V}\text{最小}_{k=1}{}^{m_k}, {}^{V}\text{最大}_{k=1} m_k\right], \forall j \in 1, V$;

同时 m_j不会发生下溢或上溢现象。

- d_j is also bounded: $1 \leq d_j \leq j, \forall j \in 1, V$. It can be easily proven by induction. The 32-bit floating point storage for d_j guarantees processing of up to $1.7 * 10^{37}$ elements in vector x without overflow. It is a reasonably large amount, but if your vector is even larger you need to use the 64-bit floating point storage for d_j.

The algorithm 2 provides the same guarantees: $1 \leq d_j \leq j, \forall j \in 1, V$.

In the remainder of this paper we will call algorithm 3 "Online Softmax".

3.1 Parallel online normalizer calculation

The lines 1-6 of the algorithm 3 define a sequential way of calculating the normalization term in a single pass over input vector. Modern computing devices allow running multiple threads concurrently; We need to have a parallel version of the algorithm to fully utilize devices. We define a generalized version of the online normalizer calculation:

$$\begin{bmatrix} m_V \\ d_V \end{bmatrix} = \begin{bmatrix} x_1 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} x_2 \\ 1 \end{bmatrix} \oplus ... \oplus \begin{bmatrix} x_V \\ 1 \end{bmatrix} \tag{3}$$

where $x_i, m_V, d_V \in \mathbb{R}$. The binary operation $\oplus : \mathbb{R}^2 \times \mathbb{R}^2 \to \mathbb{R}^2$ is defined as:

$$\begin{bmatrix} m_i \\ d_i \end{bmatrix} \oplus \begin{bmatrix} m_j \\ d_j \end{bmatrix} = \begin{bmatrix} \max(m_i, m_j) \\ d_i \times e^{m_i - \max(m_i, m_j)} + d_j \times e^{m_j - \max(m_i, m_j)} \end{bmatrix} \tag{4}$$

Applying (3) sequentially from left to right is equivalent to running lines 1-6 of the algorithm 3. The operation $\oplus$ is associative, which enables parallel evaluation of (3). It is also commutative, which provides the flexibility needed to make parallel implementations more efficient. We omit the proofs for these two statements for brevity.

4 Softmax and top-k fusion

Online Softmax (algorithm 3) does three memory accesses per vector element: one load for the normalizer calculation, one load and one store for computing Softmax function values y_i. Inference with the beam search for auto-regressive models has TopK following Softmax, and this TopK doesn't need to compute all y_i values. This enables even bigger improvements.

The TopK function is producing the vector of K integer indices referencing the largest values in the input vector, along with those values:

$$TopK\left(y\right) = (v, z) : v_i = y_{z_i}, v_i \geq y_j, \forall i \in [1, K], \forall j \notin z \tag{5}$$

where $y \in \mathbb{R}^V, z \in \mathbb{Z}^K, v \in \mathbb{R}^K$.

The TopK needs to load each element of the input vector at least once. Running Safe Softmax and the TopK separately requires 5 accesses per input element and 4 accesses if we use Online Softmax instead of Safe Softmax (but still run them separately, one after another). If we improve on the algorithm 3 and keep not only running values of m and d (when iterating over the input vector), but also the vectors of TopK input values u and their indices p - as in the algorithm 4 - we can run this Softmax+TopK fusion with just one memory access per element of the input vector.

5 Benchmarking

Online normalizer calculation reduces the number of memory accesses for the Softmax and Softmax+TopK functions. The softmax function has a very low flops per byte ratio; that means the memory bandwidth should be limiting the performance, even for Online Softmax with its additional few floating point operations per element. Fewer memory accesses should translate into performance improvements, and experiments confirm this.

We implemented a benchmark for GPUs using CUDA C. The benchmark utilizes CUB v1.8.0 for fast parallel reductions. All experiments were run on NVIDIA Tesla V100 PCIe 16 GB,

- d_j 也受到一定限制： $1 \leq d_j \leq j$、 $\forall j \in 1$、 V。这一点可以通过归纳法轻松证明。 d_j 所采用的32位浮点数存储格式能够确保在向量 x 中处理多达 $1.7 * 10^{37}$ 个元素时不会出现上溢问题。这个数值已经相当大了，但如果向量规模更大，则需要为 d_j 使用64位浮点数存储格式。

算法2同样能够提供相同的保证： $1 \leq d_j \leq j$、 $\forall j \in 1$、 V。

在本文的后续内容中，我们将把算法3称为“在线Softmax函数”。

3.1 并行在线归一化因子计算

算法3的第1至6行定义了一种通过单次遍历输入向量来顺序计算归一化项的方法。现代计算设备支持同时运行多个线程；为了充分利用这些设备，我们需要该算法的并行版本。我们由此定义了在线归一化因子计算的通用形式：

$$\begin{bmatrix} m_V \\ d_V \end{bmatrix} = \begin{bmatrix} x_1 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} x_2 \\ 1 \end{bmatrix} \oplus ... \oplus \begin{bmatrix} x_V \\ 1 \end{bmatrix} \tag{3}$$

其中包含 x_i、 m_V、 $d_V \in \mathbb{R}$。二元运算 $\oplus$： $\mathbb{R}^2 \times \mathbb{R}^2 \to \mathbb{R}^2$ 的定义如下：

$$\begin{bmatrix} m_i \\ d_i \end{bmatrix} \oplus \begin{bmatrix} m_j \\ d_j \end{bmatrix} = \begin{bmatrix} \max(m_i, m_j) \\ d_i \times e^{m_i - \max(m_i, m_j)} + d_j \times e^{m_j - \max(m_i, m_j)} \end{bmatrix} \tag{4}$$

从左到右依次应用公式(3)等同于执行算法3中的第1至6行代码。运算 $\oplus$ 具有结合性，这使得可以并行计算公式(3)。同时它还具有交换性，这一特性为提升并行实现的效率提供了便利。为简洁起见，此处不再赘述这两点性质的证明。

4 Softmax函数与top-k算法的融合

在线Softmax（算法3）每个向量元素需要执行三次内存访问：一次用于归一化因子的计算，另一次加载操作和一次存储操作则用于计算Softmax函数的数值 y_i。而用于自回归模型束搜索的推理过程会在Soft max之后应用TopK算法，且该TopK算法无需计算所有的 y_i 数值。这样一来就能实现更显著的性能提升。

TopK函数会生成一个包含K个整数索引的向量，这些索引指向输入向量中数值最大的位置，同时还会附带这些相应的数值。

$$TopK\left(y\right) = (v, z) : v_i = y_{z_i}, v_i \geq y_j, \forall i \in [1, K], \forall j \notin z \tag{5}$$

其中包含 $y \in \mathbb{R}^V$、 $z \in \mathbb{Z}^K$、 $v \in \mathbb{R}^K$。

TopK算法需要至少读取一次输入向量的每个元素。如果分别运行安全软缩放算法与TopK算法，每个输入元素需要5次内存访问；而若改用在线软最大化算法代替安全软缩放算法（依然保持分别依次运行的方式），则每个元素仅需4次访问。如果我们对算法3进行改进，像算法4那样在遍历输入向量时不仅保留 m 和 d 的当前值，还保存TopK算法所需的输入向量 u 及其索引 p，那么这种Softmax+与TopK的融合方案就能让每个输入元素仅需1次内存访问即可完成计算。

5 Benchmarking

在线归一化因子计算能够减少Softmax函数以及Softmax+TopK算法所需的内存访问次数。由于Softmax函数的每字节浮点运算次数比非常低，即便在线软最大化算法中每个元素仅需进行少量额外的浮点运算，性能瓶颈仍很可能来自内存带宽。而更少的内存访问次数有望提升性能，实验结果也证实了这一点。

我们使用CUDA C语言为GPU实现了一个基准测试。该基准测试借助CUB v1.8.0来实现快速的并行求和操作。所有实验均在配备16GB内存的NVIDIA Tesla V100 PCIe显卡上运行。

Algorithm 4 Online softmax and top-k

$m_0 \leftarrow -\infty$
$d_0 \leftarrow 0$
$u \leftarrow \{-\infty, -\infty, \ldots, -\infty\}^T, u \in \mathbb{R}^{K+1}$ ▷ The 1st K elems will hold running TopK values
$p \leftarrow \{-1, -1, \ldots, -1\}^T, p \in \mathbb{Z}^{K+1}$ ▷ ... and their indices
for $j \leftarrow 1, V$ **do**
  $m_j \leftarrow \max(m_{j-1}, x_j)$
  $d_j \leftarrow d_{j-1} \times e^{m_{j-1} - m_j} + e^{x_j - m_j}$
  $u_{K+1} \leftarrow x_j$ ▷ Initialize $K+1$ elem with new value from input vector
  $p_{K+1} \leftarrow j$ ▷ ... and its index
  $k \leftarrow K$ ▷ Sort u in descending order, permuting p accordingly. The first K elements are already sorted, so we need just a single loop, inserting the last element in the correct position.
  while $k \geq 1$ and $u_k < u_{k+1}$ **do**
    swap(u_k, u_{k+1})
    swap(p_k, p_{k+1})
    $k \leftarrow k - 1$
  end while
end for
for $i \leftarrow 1, K$ **do** ▷ The algorithm stores only K values and their indices
  $v_i \leftarrow \frac{e^{u_i - m_V}}{d_V}$
  $z_i \leftarrow p_i$
end for

ECC on, persistent mode on, CUDA Toolkit 9.1. Source code of the benchmark is available at github.com/NVIDIA/online-softmax.

5.1 Benchmarking softmax

We benchmarked all 3 Softmax algorithms - Naive, Safe, and Online - on different vector sizes for the batch sizes of 4,000 and 10. The large batch case corresponds to the training or batch inference with enough input vectors to saturate the device and and the small batch case corresponds to online inference with too few vectors to occupy the device fully.

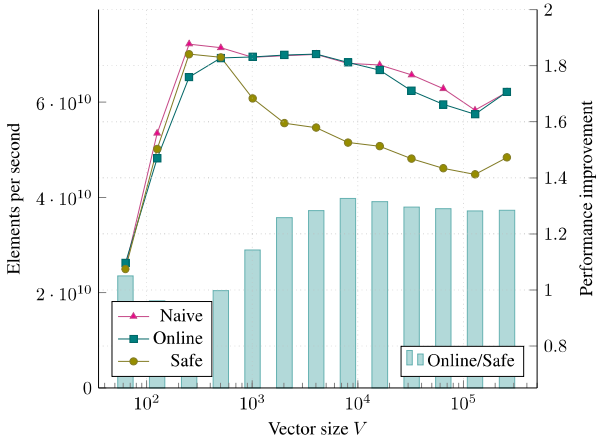


Figure 1: Benchmarking softmax, Tesla V100, fp32, batch size 4000 vectors

Algorithm 4 Online softmax and top-k

$m_0 \leftarrow -\infty$
$d_0 \leftarrow 0$
$u \leftarrow \{-\infty, -\infty, \ldots, -\infty\}^T, u \in \mathbb{R}^{K+1}$ ▷ The 1st K elems will hold running TopK values
$p \leftarrow \{-1, -1, \ldots, -1\}^T, p \in \mathbb{Z}^{K+1}$ ▷ ... and their indices
for $j \leftarrow 1, V$ **do**
  $m_j \leftarrow \max(m_{j-1}, x_j)$
  $d_j \leftarrow d_{j-1} \times e^{m_{j-1} - m_j} + e^{x_j - m_j}$
  $u_{K+1} \leftarrow x_j$ ▷ Initialize $K+1$ elem with new value from input vector
  $p_{K+1} \leftarrow j$ ▷ ... and its index
  $k \leftarrow K$ ▷ Sort u in descending order, permuting p accordingly. The first K elements are already sorted, so we need just a single loop, inserting the last element in the correct position.
  while $k \geq 1$ and $u_k < u_{k+1}$ **do**
    swap(u_k, u_{k+1})
    swap(p_k, p_{k+1})
    $k \leftarrow k - 1$
  end while
end for
for $i \leftarrow 1, K$ **do** ▷ The algorithm stores only K values and their indices
  $v_i \leftarrow \frac{e^{u_i - m_V}}{d_V}$
  $z_i \leftarrow p_i$
end for

已启用纠错码功能及持久模式，使用的CUDA工具包版本为9.1。该基准测试的源代码可在github.com/NVIDIA/online-softmax页面上获取。

5.1 Benchmarking Softmax函数

我们对批量大小为4,000和10的不同向量长度，对三种Softmax算法——即朴素算法、安全软缩放算法以及在线软最大化算法——进行了性能测试。大批量场景指的是训练或批量推理时输入向量数量充足，足以让设备达到饱和状态；而小批量场景则对应于在线推理时输入向量数量过少，无法使设备完全被利用。

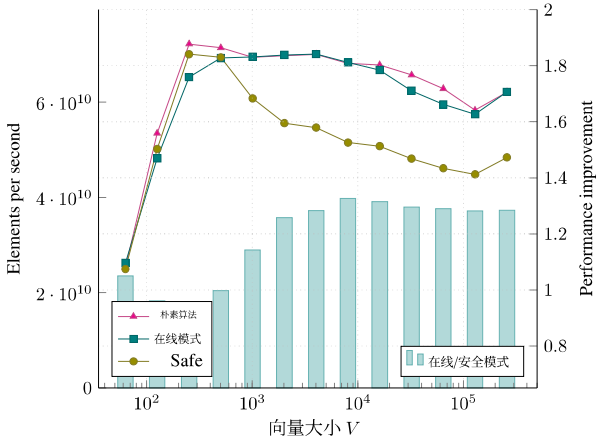


图1：Softmax函数在Tesla V100芯片上采用32位浮点格式、对4000个向量进行批量处理时的性能测试结果

For the large batch case (see figure 1) all three algorithms perform similarly up until $V = 1000$ vector size. The NVIDIA Visual Profiler shows that at that point L1 and L2 cache thrashing starts to make all three algorithms limited by the DRAM bandwidth. When this happens Online and Naive algorithms are getting faster than Safe one, quickly achieving ~ 1.3x at $V = 4000$ (look for bars in the chart, they are showing performance improvement of Online Softmax over Safe Softmax). This is quite close to 1.33x reduction in memory accesses for those algorithms.

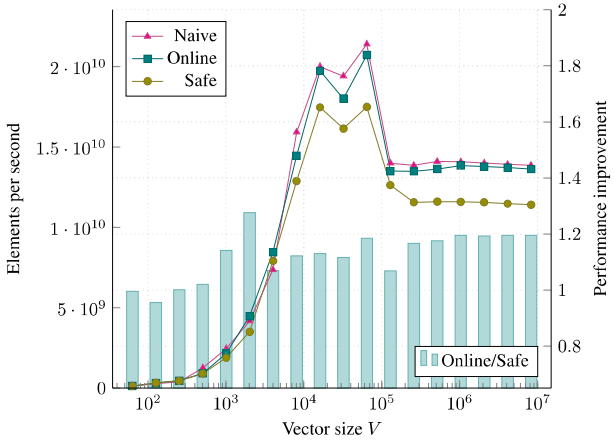


Figure 2: Benchmarking softmax, Tesla V100, fp32, batch size 10 vectors

The absolute performance for small batch case is lower for all algorithms, see figure 2. The benchmark is running one threadblock per vector; thus small batch case - with 10 vectors - has just 10 threadblocks in the grid. This is not enough to saturate the GPU, both compute and the memory subsystem are underutilized, various latencies are exposed. As in the batch inference case, all three algorithms show similar performance up to $V = 1000$ vector size. After that Naive and Online algorithms outperform Safe one by ~ 1.15x.

5.2 Benchmarking softmax and top-k

We benchmarked Safe Softmax followed by the TopK (running one after another), Safe Softmax fused with the TopK into a single function, and Online Softmax fused with TopK, again, for 2 cases: 4,000 and 10 vectors. We picked up $K = 5$ in TopK for all runs.

Online fused version is running considerably faster than Safe unfused one. For large batch case - see figure 3 - the performance improvement starts at 1.5x and goes up as vector size V increases approaching 5x at $V = 25000$, which corresponds to 5x reduction in memory accesses. This 5x comes from 2.5x due to function fusion and 2x due to Online Softmax itself.

In the small batch case (see figure 4) Online fused version outperforms Safe unfused one by 1.5x-2.5x. It cannot achieve 5x because the GPU is underutilized and the performance is limited not by the memory bandwidth, but by various latencies. Yet the reduction in memory accesses helps even in this latency limited case. In small batch case fusion only already brings substantial performance improvements, switching to Online Softmax helps improve performance even further.

The benchmark shows these levels of performance improvement for relatively small K only. The cost of keeping partial TopK results - as in the lines 10-15 of the algorithm 4 - increases quickly as K gets bigger: the performance improvement drops to 3.5x for $K = 10$, 2x for $K = 15$, 1.4x for $K = 30$, and degrades further for bigger Ks. For these cases the TopK is dominating (in terms of runtime) over the Softmax. Getting rid of separate Softmax and fusing the normalization term

在大批量场景下（见图1），在 $V = 1000$向量长度之前，三种算法的运行表现相当。NVIDIA视觉分析工具显示，达到该长度后，L1和L2缓存的高频访问开始成为限制，使得这三种算法都受到DRAM带宽的限制。此时，在线软最大化算法和朴素算法的运算速度会快于安全软缩放算法，并在 $V = 4000$ 时迅速实现 ~ 1.3倍的性能提升（可观察图表中的条形图，它们展示了在线软最大化算法相比安全软缩放算法的性能优势）。这意味着这两种算法的内存访问次数大约减少了1.33倍。

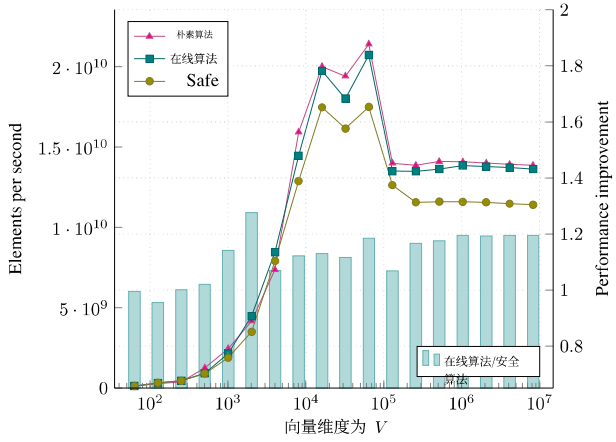


图2：Softmax函数性能测试，测试环境为Tesla V100芯片、32位浮点格式以及批量大小为10个向量的场景

如图2所示，在小批量场景下所有算法的绝对性能均较低。本次测试采用每个向量对应一个线程块的方式运行，因此包含10个向量的小批量场景在计算网格中仅包含10个线程块，这样的配置不足以让GPU满负荷运转，导致计算单元和内存子系统都未能得到充分利用，进而出现了各种延迟问题。与批量推理场景类似，在向量维度低于 $V = 1000$ 时，三种算法的性能表现较为接近；而超过该维度后，朴素算法和在线算法的性能会比安全算法高出 ~ 1.15倍。

5.2 softmax与top-k算法的基准测试

我们对三种不同组合进行了基准测试：依次运行的安全软缩放算法与TopK算法、合并为单一函数的安全软缩放算法与TopK算法，以及在线软最大化算法与TopK算法；测试场景分别为4,000个向量与10个向量的两种情况。在所有测试中，TopK算法均使用了 $K = 5$ 作为输入参数。
在线融合版本的运行速度明显快于未融合的安全算法版本。在大批量场景下——详见图3——性能提升幅度至少为1.5倍，且随着向量大小 V 的增大而进一步提升，在 $V = 25000$时可达5倍，这意味着内存访问次数相应减少了5倍。这一5倍的提升中，2.5倍源自函数融合，另外2倍则来自在线Softmax函数本身。

在小型批量场景中（见图4），在线融合版本的性能比未融合的安全算法版本高出1.5倍至2.5倍。由于其GPU利用率不足，该版本无法实现5倍的性能提升，且限制性能的并非内存带宽，而是各种延迟问题。即便在存在延迟限制的情况下，减少内存访问次数依然能带来帮助。在小型批量场景中，仅采用融合技术就已经能实现显著的性能提升，而转而使用在线软最大化算法则能让性能进一步提升。

基准测试显示，仅在 K 规模相对较小的情况下才能获得如此高的性能提升。而如算法4的第10至15行所示，若保留部分TopK结果，随着 K 规模的增大，其成本会迅速上升：当 $K = 10$为该规模时性能提升降至3.5倍，$K = 15$ 时为2倍，$K = 30$时为1.4倍，对于更大的 K规模，性能提升还会进一步下降。在这些情况下，从运行时间的角度来看，TopK算法的优势明显优于Softmax函数。通过取消独立的Softmax计算并将归一化项合并，仍然能够带来一定的好处，但随着TopK计算所需时间的不断增加，其带来的收益也会逐步降低。

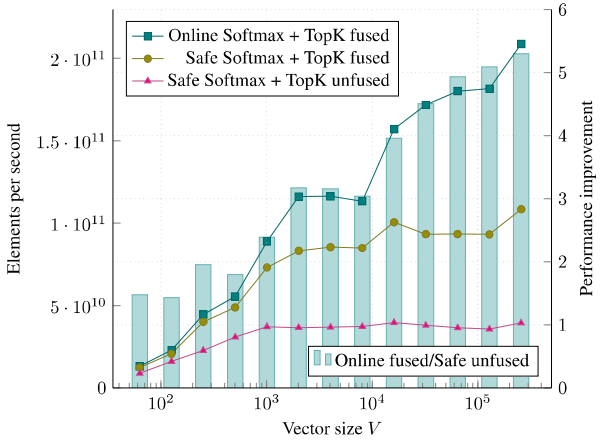


Figure 3: Benchmarking softmax and top-k, Tesla V100, fp32, batch size 4000 vectors

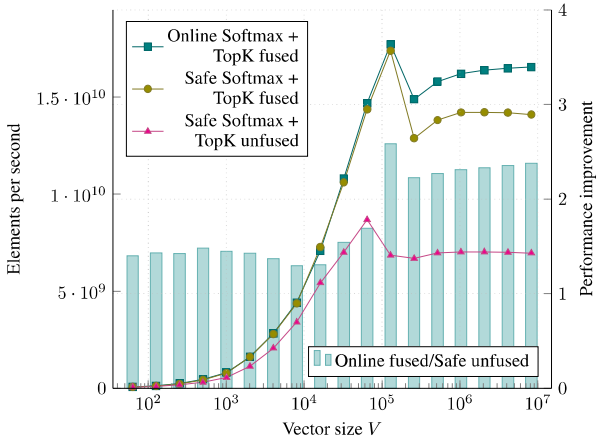


Figure 4: Benchmarking softmax and top-k, Tesla V100, fp32, batch size 10 vectors

calculation into the TopK is still beneficial, but the value goes down as TopK is taking more and more time.

6 Results

We introduced the way to calculate the normalizer for the Softmax function in a single pass over input data, which reduces memory accesses by 1.33x for the Softmax function alone. Benchmarks

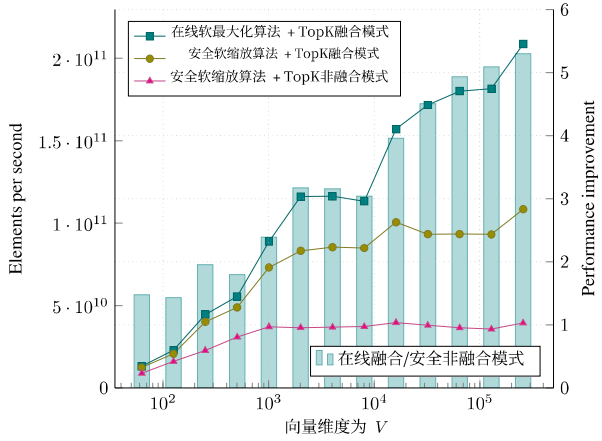


图3：Softmax函数与TopK算法的基准测试，使用Tesla V100芯片，采用32位浮点格式处理4000个向量的批量数据

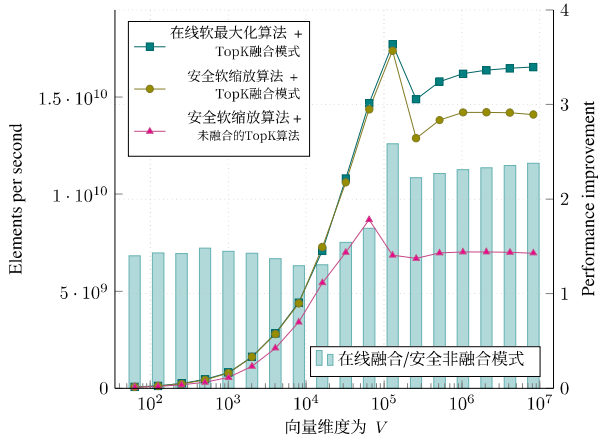


图4：在Tesla V100芯片、32位浮点格式以及批量大小为10个向量的场景下，对Softmax函数与top-k算法进行的性能测试

将归一化计算整合到TopK算法中依然有益，不过随着TopK计算耗时的增加，其带来的收益也在逐渐减少。

6个测试结果

我们提出了一种仅需对输入数据进行单次遍历即可计算Softmax函数归一化因子的方法，该方法仅针对Softmax函数本身就能将内存访问次数减少1.33倍。相关基准测试表明，

on Tesla V100 show that this materializes in 1.15x performance improvements for $V \geq 1000$ vector sizes, and for the large batch mode it goes up to 1.3x when $V \geq 4000$.

If one is using Naive Softmax then switching to Online version improves numerical accuracy with no performance hit or a negligible one.

When the TopK follows the Softmax the new single-pass normalizer calculation enables efficient fusion of these 2 functions resulting in 5x fewer memory accesses for Softmax+TopK combined. We observed 1.5x-5x performance improvement on Tesla V100, with this 5x improvement coming from 2.5x with fusion and 2x with Online Softmax itself.

These performance improvements could be applied not only to the classical Softmax function; They are orthogonal to many other Softmax optimization techniques including Hierarchical Softmax, Importance Sampling, and SVD-Softmax.

7 Discussion

Online Softmax is running up to 1.3x faster on the latest generation GPU than the one used by major DL frameworks. It also enables very efficient fusion of the Softmax with following TopK showing up to 5x performance improvement over the traditional Safe Softmax and TopK running separately.

Could we see significantly different speed-ups or even slow-downs on different compute devices, for example CPUs? We didn't do experiments for those, but if the original code is vectorized and one manages to keep it vectorized for the online normalizer (and partial TopK) calculation then similar speedups could probably be expected.

There could be a way to improve the performance further. The resulting Softmax and even Softmax+TopK fused are still limited by the memory bandwidth, so fusing them with the preceding layer will avoid memory round trip, thus improving performance. This change is more challenging though.

Acknowledgments

We would like to thank Christoph Angerer for his valuable comments and suggestions.

References

[1] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to Sequence Learning with Neural Networks. *ArXiv e-prints*, September 2014, 1409.3215.

[2] G. Hinton, L. Deng, D. Yu, G. E. Dahl, A. r. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. N. Sainath, and B. Kingsbury. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine*, 29(6):82–97, Nov 2012. ISSN 1053-5888.

[3] Engel J. Polytomous logistic regression. *Statistica Neerlandica*, 42(4):233–252.

[4] W. Chen, D. Grangier, and M. Auli. Strategies for Training Large Vocabulary Neural Language Models. *ArXiv e-prints*, December 2015, 1512.04906.

[5] Kyuhong Shim, Minjae Lee, Iksoo Choi, Yoonho Boo, and Wonyong Sung. Svd-softmax: Fast softmax approximation on large vocabulary neural networks. In *Advances in Neural Information Processing Systems 30*, pages 5463–5473. Curran Associates, Inc., 2017. URL http://papers.nips.cc/paper/7130-svd-softmax-fast-softmax-approximation-on-large-vocabulary-

[6] Joshua Goodman. Classes for fast maximum entropy training. In *ICASSP*, pages 561–564. IEEE, 2001. ISBN 0-7803-7041-4. URL http://dblp.uni-trier.de/db/conf/icassp/icassp2001.html#Goodman01.

[7] T. Mikolov, S. Kombrink, L. Burget, J. Černocký, and S. Khudanpur. Extensions of recurrent neural network language model. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5528–5531, May 2011.

在Tesla V100芯片上的测试结果显示，对于 $V \geq 1000$ 规模的向量而言，这一方法的性能提升可达1.15倍；而在大规模批量处理模式下，当 $V \geq 4000$时，性能提升幅度甚至可达到1.3倍。
如果使用的是朴素Softmax函数，那么切换到在线版本可以提高数值精度，且不会对性能造成影响或仅会产生可忽略的轻微影响。
当TopK算法与Softmax函数结合使用时，这种新的单次遍历式归一化因子计算方式能够高效地融合这两个函数，从而使Softmax+TopK组合体的内存访问次数减少5倍。我们在Tesla V100平台上观察到其性能提升了1.5倍至5倍，其中5倍的提升来自融合处理带来的2.5倍提升，以及在线软最大化算法本身带来的2倍提升。

这些性能提升不仅可应用于传统的Softmax函数；同时，它们也与包括分层Softmax、重要性采样以及SVD-Softmax在内的许多其他Softmax优化技术互不冲突。

7 讨论

在最新一代GPU上，相比主流深度学习框架所使用的硬件，在线Softmax函数的运算速度可快达1.3倍。此外，它还能实现Softmax函数与后续TopK算法的高效融合，其性能提升幅度可达传统单独运行安全软缩放算法和TopK算法时的5倍。
在不同的计算设备上，比如CPU上，性能提升或下降的幅度会不会出现显著差异？我们尚未针对这些设备进行实验，但如果原始代码已经是向量化处理的，并且能够在执行在线归一化因子计算（以及部分TopK算法）时继续保持向量化形式，那么大概也能期望得到类似的性能提升。

或许还有进一步提升性能的方法。最终得到的Softmax函数，甚至是经过Soft-max+TopK融合后的结果，依然会受到内存带宽的限制；因此将它们与前置层进行融合，可以避免内存的来回传输，进而提升性能。不过这种改动难度更大。

致谢
我们要感谢Christoph Angerer提出的宝贵意见与建议。

参考文献

[1] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to Sequence Learning with Neural Networks. *ArXiv e-prints*, September 2014, 1409.3215.

[2] G. Hinton, L. Deng, D. Yu, G. E. Dahl, A. r. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. N. Sainath, and B. Kingsbury. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine*, 29(6):82–97, Nov 2012. ISSN 1053-5888.

[3] Engel J. Polytomous logistic regression. *Statistica Neerlandica*, 42(4):233–252.

[4] W. Chen, D. Grangier, and M. Auli. Strategies for Training Large Vocabulary Neural Language Models. *ArXiv e-prints*, December 2015, 1512.04906.

[5] Kyuhong Shim, Minjae Lee, Iksoo Choi, Yoonho Boo, and Wonyong Sung. Svd-softmax: Fast softmax approximation on large vocabulary neural networks. In *Advances in Neural Information Processing Systems 30*, pages 5463–5473. Curran Associates, Inc., 2017. URL http://papers.nips.cc/paper/7130-svd-softmax-fast-softmax-approximation-

面向大规模词汇表的

[6]Joshua Goodman. 用于快速最大熵训练的类. 收录于ICASSP会议论文集，第561–564页。IEEE出版社，2001年。ISBN 0-7803-7041-4。网址：http://dblp.uni-trier.de/db/conf/icassp/icassp2001.html#Goodman01。

[7] T. Mikolov, S. Kombrink, L. Burget, J. Cernocký和S. Khudanpur. 循环神经网络语言模型的扩展. 收录于2011年IEEE国际声学、语音与信号处理会议（ICASSP）论文集，第5528–5531页，2011年5月。

[8] E. Grave, A. Joulin, M. Cissé, D. Grangier, and H. Jégou. Efficient softmax approximation for GPUs. *ArXiv e-prints*, September 2016, 1609.04309.

[9] Yoshua Bengio and Jean-Sébastien Sénécal. Quick training of probabilistic neural nets by importance sampling. In *Proceedings of the conference on Artificial Intelligence and Statistics (AISTATS)*, 2003.

[10] A. Mnih and Y. Whye Teh. A Fast and Simple Algorithm for Training Neural Probabilistic Language Models. *ArXiv e-prints*, June 2012, 1206.6426.

[11] S. Ji, S. V. N. Vishwanathan, N. Satish, M. J. Anderson, and P. Dubey. BlackOut: Speeding up Recurrent Neural Network Language Models With Very Large Vocabularies. *ArXiv e-prints*, November 2015, 1511.06909.

[12] Jacob Devlin, Rabih Zbib, Zhongqiang Huang, Thomas Lamar, Richard Schwartz, and John Makhoul. Fast and robust neural network joint models for statistical machine translation. In *Proceedings of ACL2014*, pages 1370–1380, 2014.

[13] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL `https://www.tensorflow.org/`. Software available from tensorflow.org.

[14] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. In *NIPS-W*, 2017.

[15] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Twenty-ninth Annual Conference on Neural Information Processing Systems (NIPS)*, 2015. URL `https://github.com/dmlc/web-data/raw/master/mxnet/paper/mxnet-learningsys.pdf`.

[16] Frank Seide and Amit Agarwal. Cntk: Microsoft's open-source deep-learning toolkit. In *Proceedings of the 22Nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pages 2135–2135, New York, NY, USA, 2016. ACM. ISBN 978-1-4503-4232-2. URL `http://doi.acm.org/10.1145/2939672.2945397`.

[17] Seiya Tokui, Kenta Oono, Shohei Hido, and Justin Clayton. Chainer: a next-generation open source framework for deep learning. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Twenty-ninth Annual Conference on Neural Information Processing Systems (NIPS)*, 2015. URL `http://learningsys.org/papers/LearningSys_2015_paper_33.pdf`.

[18] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962. URL `https://amstat.tandfonline.com/doi/abs/10.1080/00401706.1962.10490022`.

[8] E. Grave、A. Joulin、M. Cissé、D. Grangier以及H. Jégou所著。《面向GPU的高效Softmax函数近似算法》。ArXiv预印本，2016年9月，编号1609.04309。[9] Yoshua Bengio与Jean-Sébastien Sénécal合著。《通过重要性采样实现概率神经网络的高速训练》。载于2003年人工智能与统计学会议（AISTATS）论文集。[10] A. Mnih与Y. Whye Teh所著。《一种快速且简单的神经概率语言模型训练算法》。ArXiv预印本，2012年6月，编号1206.6426。

[11] S. Ji, S. V. N. Vishwanathan, N. Satish, M. J. Anderson 和 P. Dubey。BlackOut：利用超大词汇量加速循环神经网络语言模型的运行。ArXiv预印本，2015年11月，1511.06909。

[12] Jacob Devlin、Rabih Zbib、Zhongqiang Huang、Thomas Lamar、Richard Schwartz以及John Makhoul。用于统计机器翻译的快速且稳健的神经网络联合模型。载于《ACL2014会议论文集》，第1370–1380页，2014年。

[13] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL `https://www.tensorflow.org/`. Software available from tensorflow.org.

[14] Adam Paszke、Sam Gross、Soumith Chintala、Gregory Chanan、Edward Yang、Zachary DeVito、Zeming Lin、Alban Desmaison、Luca Antiga以及Adam Lerer。PyTorch中的自动微分技术。载于2017年神经信息处理系统年会论文集NIPS-W。

[15]陈天奇、李沐、李宇天、林敏、王乃岩、王敏杰、肖天军、徐冰、张驰远以及张铮。Mxnet：一种适用于异构分布式系统的灵活高效的机器学习库。载于2015年第二十九届神经信息处理系统年会（NIPS）期间的机器学习系统研讨会（LearningSys）论文集。网址：https://github.com/dmlc/web-data/raw/master/mxnet/paper/mxnet-learningsys.pdf。

[16] Frank Seide and Amit Agarwal. Cntk: Microsoft's open-source deep-learning toolkit. In *Proceedings of the 22Nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pages 2135–2135, New York, NY, USA, 2016. ACM. ISBN 978-1-4503-4232-2. URL `http://doi.acm.org/10.1145/2939672.2945397`.

[17] Seiya Tokui, Kenta Oono, Shohei Hido, and Justin Clayton. Chainer: a next-generation open source framework for deep learning. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Twenty-ninth Annual Conference on Neural Information Processing Systems (NIPS)*, 2015. URL `http://learningsys.org/papers/LearningSys_2015_paper_33.pdf`.

[18] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962. URL `https://amstat.tandfonline.com/doi/abs/10.1080/00401706.1962.10490022`.